



Programmiertechnik II

Gerichtete Graphen

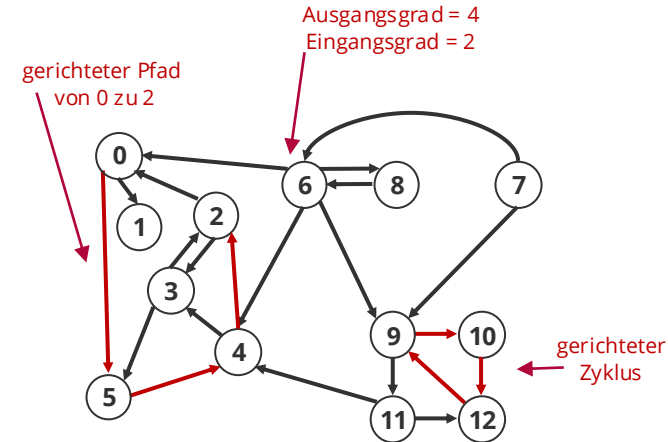
Ralf Herbrich

1. Begriffe
2. Zyklen
3. Topologisches Sortieren
4. Zusammenhängende Komponenten

1. **Begriffe**
2. Zyklen
3. Topologisches Sortieren
4. Zusammenhängende Komponenten

Gerichtete Graphen

- **Definition (Graph).** Ein **Graph** $G = (V, E)$ besteht aus einer Menge V von Knoten (*vertices*, *nodes*) und einer Menge $E \subseteq V \times V$ von Kanten.
- **Definition (Gerichteter Graph).** Ein Graph $G = (V, E)$ ist ungerichtet falls $\forall (v, v') \in E \rightarrow (v', v) \in E$. Andernfalls sprechen wir von einem gerichteten Graphen (*digraph*).
- **Definition (Pfad).** Eine Folge von Kanten $e_1, e_2, \dots, e_n$ heißt **gerichteter Pfad der Länge n** genau dann wenn für alle i : $e_i = (v', v)$, $e_{i+1} = (v, v'')$.
- **Definition (Zyklus).** Ein Pfad $e_1, e_2, \dots, e_n$ heißt **zyklisch** genau dann wenn $e_{1,1} = e_{n,2}$ (das heißt, der erste und letzte Knoten im Pfad sind identisch).
- **Definition (Verbundenheit).** Zwei Knoten u und v sind **verbunden** wenn ein Pfad $e_1, e_2, \dots, e_n$ existiert so dass $e_1 = (u, v')$ und $e_n = (v'', v)$.



digraph.h

Programmiertechnik II

Unit 9b – Gerichtete Graphen

Gerichtete Graphen in der echten Welt

■ Gerichtete Graphen in der echten Welt:

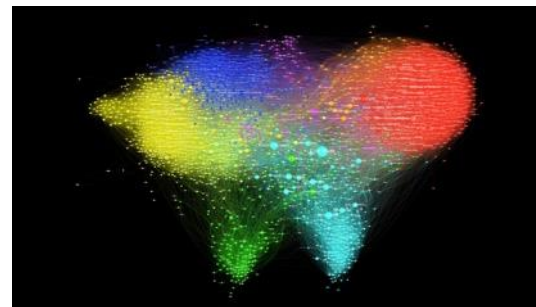
- Straßennetz (Einbahnstraßen)
- Webgraph (Links)
- Follower-Graph ([Twitter](#))

■ Probleme in gerichteten Graphen:

1. Zyklen finden (*directed cycle*)
2. Knoten sortieren (*topological sort*)
3. Verbundenheit (*connectivity*)

■ Pfade finden in gerichteten Graphen ist identisch zu ungerichteten Graphen (Tiefen- und Breitensuche).

- **Beispiel:** Web-Crawler (Breitensuche!)



Programmiertechnik II

Unit 9b – Gerichtete Graphen

Gerichtete Graphen in der echten Welt (*ctd*)

- Gerichtete Graphen sind eine perfekte Abstraktion für gerichtete Interaktionen von physikalischen Entitäten und tauchen ständig im Alltag auf.

Graph	Knoten	Kanten
Transport	Kreuzung	Einbahnstraße
Internet	Webseite	Hyperlinks
Soziales Netzwerk	Person	Folgen
Ernährungskette	Spezies	Raubtier-Beute Beziehung
Epidemiologie	Person	Infektion
Objektgraph	Objekt	Zeiger
Planung	Aufgabe	Vorgänger Bedingung
Brettspiel	Stellung	Legal Zug

Programmiertechnik II

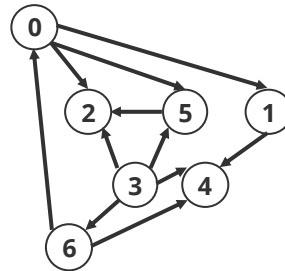
Unit 9b – Gerichtete Graphen

1. Begriffe
2. **Zyklen**
3. Topologisches Sortieren
4. Zusammenhängende Komponenten

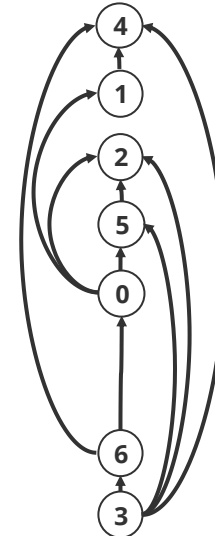
- **Ziel:** Gegeben eine Menge von Aufgaben unter vorgegebenen Vorrangbedingungen, in welcher Reihenfolge müssen diese Aufgaben ausgeführt werden, um alle Vorrangbedingungen zu erfüllen?
- **Digraph Model:** Knoten = Task, Kante = Vorrangbedingung

0. Softwaretechnik
1. Komplexitätstheorie
2. Künstliche Intelligenz
3. Programmiertechnik I
4. Theoretische Informatik I
5. Wissenschaftliches Rechnen
6. Programmiertechnik II

Aufgabenliste

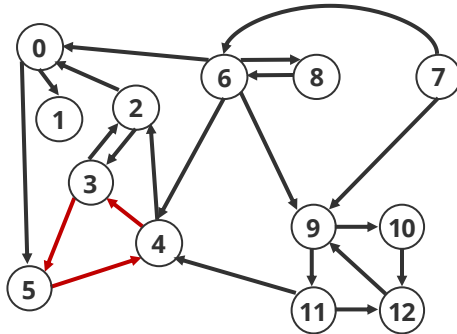


Vorrangbedingungsgraph

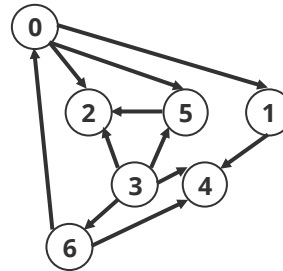


Mögliche Lösung

- **Beobachtung 1:** Ein Graph mit Vorrangbedingungen muss azyklisch (*acyclic*) sein, um eine Reihenfolge zu finden, in der die Aufgaben erledigt werden können.
 - **Problem:** Ein Graph kann eine exponentielle Anzahl von Zyklen haben.
- **Beobachtung 2:** Wenn wir in einer Tiefensuche von einem Knoten $w \in V$ aus an einem Knoten $v \in V$ auf dem Stack ankommen, der eine Kante $(v, w) \in E$ hat, dann gibt es mindestens einen Zyklus der w enthält.



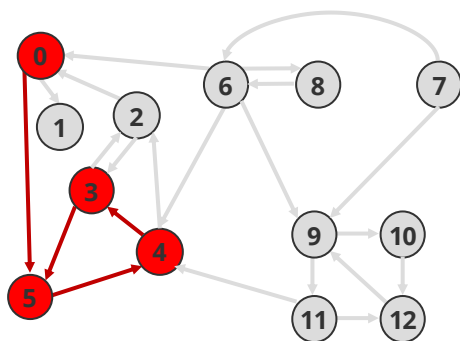
Digraph mit Zyklen



Azyklischer Digraph

Zyklen Finden

- **Idee:** Wenn Tiefensuche von jedem Knoten aus keinen Zyklus hat, dann ist der Digraph zyklensfrei!
 - `marked` markiert schon besuchte Knoten
 - `edge_to[w] == v` bedeutet, dass die Kante nach `w` von `v` kommt
 - `on_stack[w]` markiert, dass `w` auf dem momentanen Pfad liegt



v	marked	edge_to	on_stack
0	T	-	T
1	F	-	F
2	F	-	F
3	T	4	T
4	T	5	T
5	T	0	T
6	F	-	F
7	F	-	F
8	F	-	F
9	F	-	F
10	F	-	F
11	F	-	F
12	F	-	F

```
// run DFS and find a directed cycle (if one exists)
void dfs(const Digraph& dg, const int v) {
    on_stack[v] = true;
    marked[v] = true;
    for (auto w : dg.adj(v)) {
        // short circuit if directed cycle found
        if (dcycle != nullptr) {
            return;
        } else if (!marked[w]) {
            // found new vertex, so recur
            edge_to[w] = v;
            dfs(dg, w);
        } else if (on_stack[w]) {
            // trace back directed cycle
            dcycle = new Stack<int>();
            for (auto x = v; x != w; x = edge_to[x])
                dcycle->push(x);
            dcycle->push(w);
            dcycle->push(v);
        }
    }
    on_stack[v] = false;
}
```

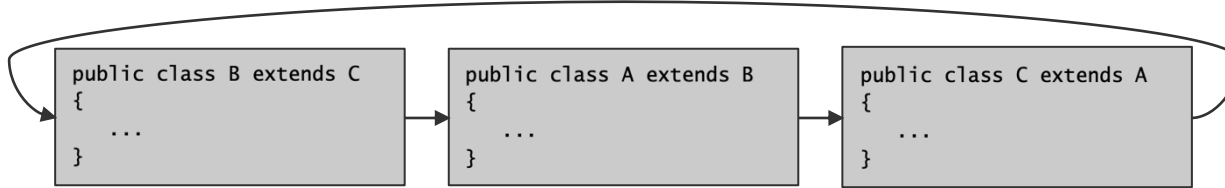
[directed_cycle.h](#)

Programmiertechnik II

Unit 9b – Gerichtete Graphen

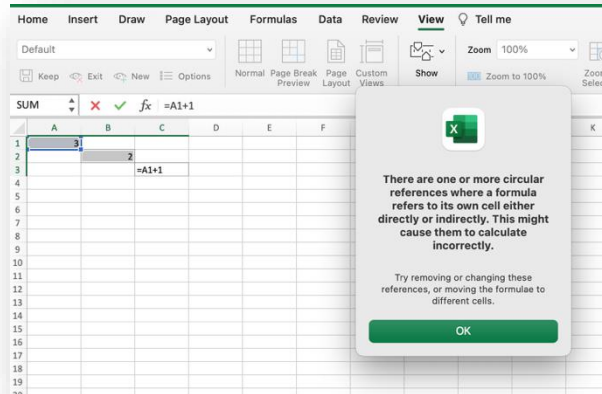
Anwendungen von Zyklen Finden

■ Java Compiler findet Zyklen in Klassendeklarationen



```
% javac A.java
A.java:1: cyclic inheritance
involving A
public class A extends B { }
        ^
1 error
```

■ Microsoft Excel findet Zyklen in Tabellenformeln

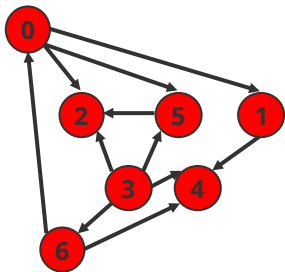


1. Begriffe
2. Zyklen
- 3. Topologisches Sortieren**
4. Zusammenhängende Komponenten

Topologisches Sortieren

- **Definition.** Gegeben einen azyklischen Digraphen (DAG) $G = (V, E)$, ist eine topologische Ordnung eine **Knotenordnung**, bei der alle **gerichteten Kanten von** einem in der Ordnung **vorhergehenden Knoten** zu einem in der Ordnung **nachfolgendem Knoten** zeigen.
- **Idee:** Tiefensuche laufen lassen und Knoten in umgekehrter Reihenfolge zurückgeben

□ marked markiert schon besuchte Knoten



v	marked
0	T
1	T
2	T
3	T
4	T
5	T
6	T

post_order 3 6 0 5 2 1 4

```
// run DFS in digraph dg from vertex v and computes postorder
void dfs(const Digraph& dg, const int v) {
    marked[v] = true;
    for (auto w : dg.adj(v)) {
        if (!marked[w]) dfs(dg, w);
    }
    post_order->push(v);
    return;
}

public:
// constructor
Topological(const Digraph& dg) {
    // initialize the marked array
    marked = new bool[dg.V()];
    for (auto v = 0; v < dg.V(); v++) marked[v] = false;
    post_order = new Stack<int>();

    // run DFS to compute the order
    for (auto v = 0; v < dg.V(); v++)
        if (!marked[v]) dfs(dg, v);

    // free the marked array and set the pointer back to nullptr
    delete[] marked;
    marked = nullptr;
}
```

topological.h

Programmiertechnik II

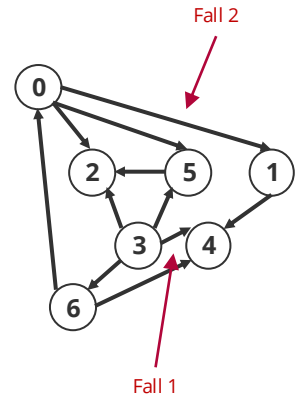
Unit 9b – Gerichtete Graphen

Topologisches Sortieren: Korrektheit

- **Satz:** Die umgekehrte Postorder-Reihenfolge in einem azyklischen Digraphen ist eine topologische Sortierung.
- **Beweis:** Betrachten wir eine beliebige Kante $(v, w) \in E$. Einer der drei Fälle von $\text{dfs}(v)$ muss zutreffen:
 1. $\text{dfs}(w)$ wurde bereits aufgerufen und ist zurückgekehrt (w ist markiert)
 2. $\text{dfs}(w)$ wurde noch nicht aufgerufen (w ist unmarkiert) so dass $(v, w) \in E$ bewirkt, dass $\text{dfs}(w)$ aufgerufen wird und zurückkehrt, bevor $\text{dfs}(v)$ zurückkehrt
 3. $\text{dfs}(w)$ wurde aufgerufen und ist noch nicht zurückgekehrt, wenn $\text{dfs}(v)$ aufgerufen wird.

Fall 3 ist aber nicht möglich, da der Digraph azyklisch war (wenn der Fall möglich ist, dann vervollständigt (v, w) einen Zyklus).

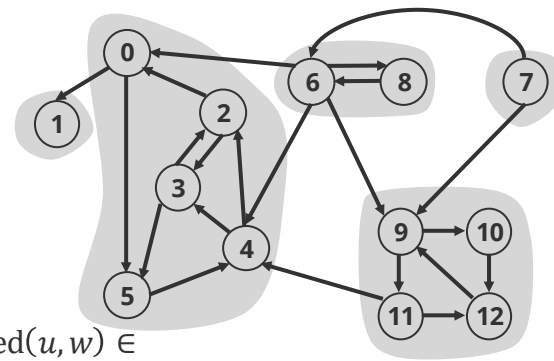
In Fall 1 und 2 erscheint aber w immer **nach** v in umgekehrter Postorder-Reihenfolge!



1. Begriffe
2. Zyklen
3. Topologisches Sortieren
4. **Zusammenhängende Komponenten**

Starke Zusammenhangskomponenten

- **Definition (Stark Zusammenhängend).** Die Knoten $v \in V$ und $w \in V$ sind **stark zusammenhängend** wenn es sowohl einen gerichteten Pfad von v zu w **und** von w zu v gibt.
- Die Relation $\text{sconnected}(v, w)$ (v ist stark zusammenhängend mit w) ist eine **Äquivalenzrelation**:
 - **Reflexivität:** $\text{sconnected}(v, v) \in R$
 - **Symmetrie:** $\text{sconnected}(v, w) \in R \Rightarrow \text{sconnected}(w, v) \in R$
 - **Transitivität:** $\text{sconnected}(u, v) \in R \wedge \text{sconnected}(v, w) \in R \Rightarrow \text{sconnected}(u, w) \in R$
- **Definition (Starke Zusammenhangskomponente):** Eine starke Zusammenhangskomponente eines Digraphen $G = (V, E)$ ist eine maximale Menge stark zusammenhängender Knoten.



5 starke Zusammenhangskomponenten

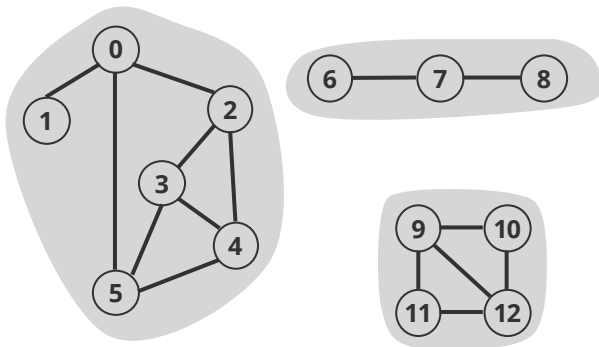
- [illegible]

Programmiertechnik II

Unit 9b – Gerichtete Graphen

(Starke) Zusammenhangskomponenten

v und w sind **zusammenhängend** wenn es einen Pfad zwischen v und w gibt

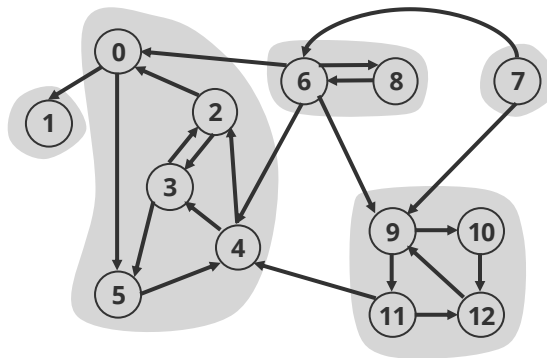


3 Zusammenhangskomponenten

node_id[]	0	1	2	3	4	5	6	7	8	9	10	11	12
cc_id[]	0	0	0	0	0	0	1	1	1	2	2	2	2

Algorithmus: Tiefensuche (DFS)

v und w sind **stark zusammenhängend** wenn es einen gerichteten Pfad zwischen v und w und einen gerichteten Pfad zwischen w und v gibt



5 starke Zusammenhangskomponenten

node_id[]	0	1	2	3	4	5	6	7	8	9	10	11	12
cc_id[]	1	0	1	1	1	1	2	3	2	4	4	4	4

Algorithmus: ?

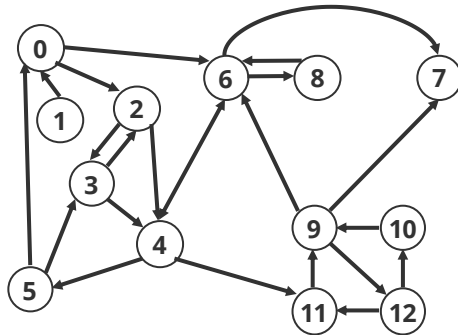
Programmiertechnik II

Unit 9b – Gerichtete Graphen

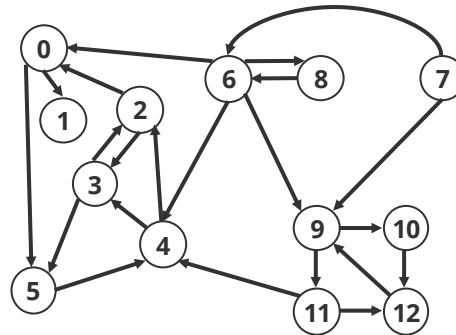
Kosaraju-Sharir Algorithmus

- Einfacher Algorithmus um starke Zusammenhangskomponenten zu berechnen
 - **Phase 1:** Tiefensuche auf inversem Digraph laufen lassen und Knoten in umgekehrter Reihenfolge zurückgeben
 - **Phase 2:** Tiefensuche auf Digraphen laufen lassen mit Knotenreihenfolge aus Phase 1

post_order 1 0 2 4 5 3 11 9 12 10 6 7 8



v	marked
0	T
1	T
2	T
3	T
4	T
5	T
6	T
7	T
8	T
9	T
10	T
11	T
12	T



v	marked	cc_id
0	T	1
1	T	0
2	T	1
3	T	1
4	T	1
5	T	1
6	T	3
7	T	4
8	T	3
9	T	2
10	T	2
11	T	2
12	T	2

```
// run DFS in digraph dg from vertex v
void dfs(const Digraph& dg, const int v) {
    marked[v] = true;
    cc_id[v] = no_components;
    for (auto w : dg.adj(v)) {
        if (!marked[w]) {
            dfs(dg, w);
        }
    }
    return;
}

public:
// constructor
KosarajuSharirSCC(const Digraph& dg) : no_vertices(dg.V()),
                                       no_components(0) {
    // initialize the marked and cc_id array
    marked = new bool[no_vertices];
    cc_id = new int[no_vertices];
    for (auto v = 0; v < no_vertices; v++) marked[v] = false;

    // run depth-first search recursively in the topological
    DepthFirstOrder dfo(dg.reverse());
    for (auto v : dfo.reverse_order()) {
        if (!marked[v]) {
            dfs(dg, v);
            no_components++;
        }
    }
}
```

sc.c.h

Programmiertechnik II

Unit 9b – Gerichtete Graphen

Beweis der Korrektheit des Kosaraju-Sharir Algorithmus

- **Satz:** Bei der Tiefensuche in einem Diagraphen G , wobei die markierten Knoten in umgekehrter Reihenfolge einer Tiefensuche in G^R betrachtet werden, liegen die Knoten, die bei jedem Aufruf der rekursiven Methode im Konstruktor erreicht werden, in einer starken Zusammenhangskomponente.
- **Beweis:**
 1. **Jeder Knoten v , der mit s in einer starken Zusammenhangskomponente liegt, wird durch den Aufruf von $\text{dfs}(G, s)$ im Konstruktor erreicht.** Nehmen wir an, v würde nicht erreicht weil v schon markiert war. Da beide stark verbunden sind, wäre dann aber s markiert und würde nicht im Konstruktor aufgerufen. Widerspruch!
 2. **Jeder Knoten v , der durch den Aufruf von $\text{dfs}(G, s)$ im Konstruktor erreicht, liegt mit s in einer starken Zusammenhangskomponente.**
 - Offensichtlich gibt es einen Pfad von s zu v , weil der Aufruf $\text{dfs}(G, s)$ v erreicht.
 - Zu zeigen: Es gibt einen Pfad von v zu s in $G \Leftrightarrow$ es gibt einen Pfad s zu v in G^R
 - Umgekehrte Reihenfolge einer Tiefensuche in G^R impliziert aber, dass es diesen Pfad gibt!

Geschichte Kosaraju-Sharir Algorithmus

- **1960:** Schlüsselproblem des *Operation Research*
 - Viele *greedy* Algorithmen wurden vorgeschlagen
 - Komplexitätsanalysen nicht durchgeführt.
- **1972:** Erster linearer Algorithmus mit Tiefensuche (Tarjan's)
 - Klassischer Algorithmus
 - Schwer verständlich aber Bedeutung von Tiefensuche
- **1980s:** Zwei-phasen Algorithmus mit Tiefensuche (Kosaraju-Sharir)
 - Klassischer Algorithmus
 - Rao Kosaraju entdeckte ihn als Teil seiner Vorlesung 1978 (aber hat ihn nie publiziert!)
 - Micha Sharir entdeckte ihn und publizierte das Verfahren 1981
 - Später wiederentdeckt in russischer Wissenschaftsliteratur von 1972 (!)
- **1990s:** Noch schnellere (fast-lineare) Algorithmen
 - Cheriyan-Mehlhorn: Ein-Pass Algorithmus der praktisch linear (aber komplex) ist



Robert Tarjan
(1948 –)



S. Rao Kosaraju
(1943 –)



Micha Sharir
(1950 –)

Programmiertechnik II

Unit 9b – Gerichtete Graphen

■ Gerichtete Graphen

- Weit verbreitet wenn es Richtungen in der Abhängigkeit zweier Entitäten gibt
- Alle Algorithmen der Erreichbarkeit (Tiefensuche & Breitensuche) funktionieren analog

■ Zyklen

- Zentrale Bedeutung bei Aufgabenplanung (*scheduling*) ist Zyklen zu erkennen
- Es gibt exponentiell viele Zyklen aber mit Tiefensuche können wir einen in linearer Zeit finden!

■ Topologisches Sortieren

- Knotenordnung bei azyklische gerichtete Graph (DAG), so dass Kanten immer „nach vorn“ zeigen
- Tiefensuche auch hier die algorithmische Lösung mit linearem Zeitaufwand

■ (Stark) zusammenhängende Komponenten

- Verallgemeinerung des Konzepts von “Verbundenheit” in gerichteten Graphen
- Lösungsalgorithmus: Tiefensuche!

Viel Spaß bis zur nächsten Vorlesung!